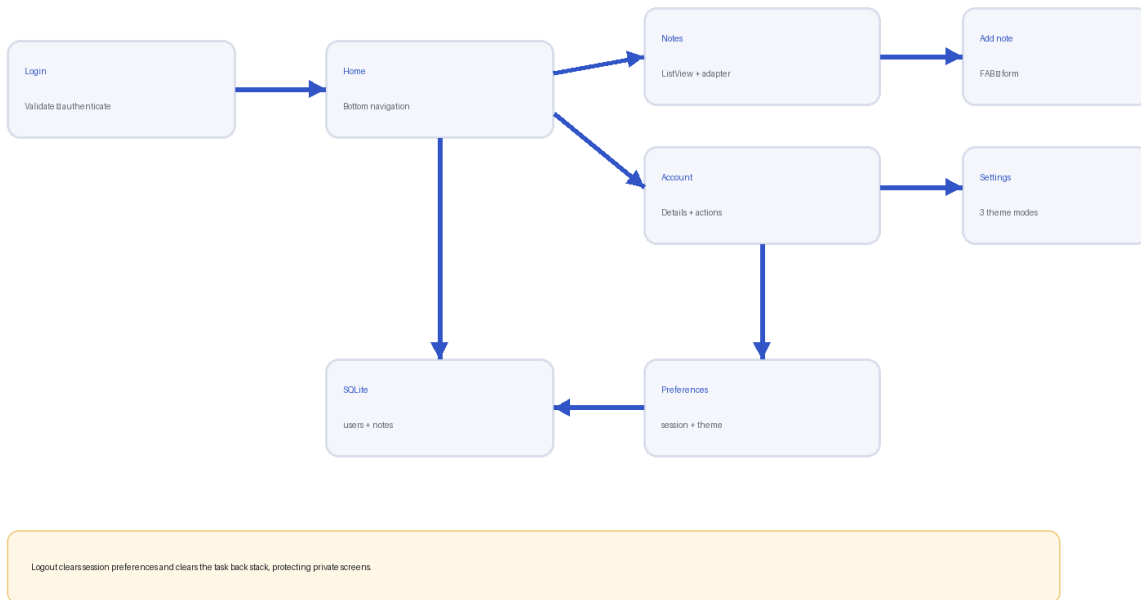


SECURE NOTES AND SETTINGS

CAT 3 Course Project Report

Secure Notes Application flow

Validation happens before SQLite access; settings remain in SharedPreferences



Unit: BSD 324 Mobile Programming II

Technology: Kotlin • Android Views • SQLite • SharedPreferences

Group members: _____

Submission date: _____

1. Executive summary

Secure Notes is a local Android application that allows a person to create an account, authenticate, manage private notes, view account details, and choose a system, light, or dark appearance. The project directly implements the CAT 3 brief using Android Studio's Empty Views Activity approach and concepts covered in the course notes: SQLiteOpenHelper, structured resources, ListView adapters, application preferences, testing, and Android project organization.

2. Objectives and requirement coverage

Requirement	Implementation
Authentication	Sign-up and login; pre-query validation; SQLite credentials; salted PBKDF2 hash.
Navigation	BottomNavigationView switches between Notes and Account fragments.
Notes	Title/subtitle ListView, custom BaseAdapter, FAB add form, long-press deletion.
Settings	SharedPreferences saves system/light/dark; privacy statement included.
Resources	Separate strings, colors, dimensions, styles, themes, menus, drawables, and animations.

3. Design decisions

The application uses Activities for independent flows (login, sign-up, adding a note, and settings) and Fragments for destinations hosted by HomeActivity. View Binding provides type-safe XML references. Database work is executed on a single-thread executor so disk access does not block the main UI thread. Material components provide consistent input errors, navigation, cards, radio buttons, and a floating action button.

4. Database structure

Secure Notes SQLite schema

Two related tables keep accounts separate from note content.



Privacy boundary: every notes query filters by user_id. ON DELETE CASCADE removes orphaned notes.

The users table owns authentication records. The notes table has a many-to-one relationship with users through user_id. Email is unique and case-insensitive. A foreign-key constraint with ON DELETE CASCADE maintains referential integrity. An index on notes.user_id speeds up per-user retrieval.

4.1 Password handling

Plain-text passwords are never inserted. Sign-up creates a 16-byte cryptographically random salt and derives a 256-bit value with PBKDF2-HMAC-SHA256 using 120,000 iterations. Login fetches the matching email record, derives a hash from the attempted password and stored salt, and compares it with the stored hash. For a production internet service, authentication should be server-side and transport should use TLS.

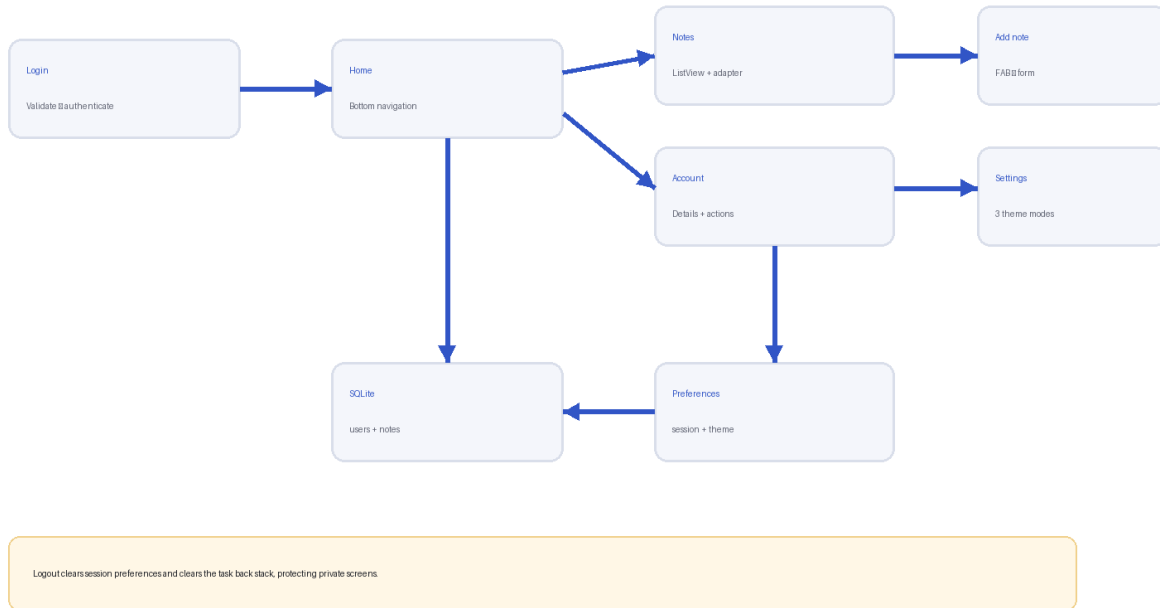
4.2 SQLiteOpenHelper lifecycle

onConfigure enables foreign keys. onCreate executes CREATE TABLE and CREATE INDEX statements. onUpgrade currently recreates the version-1 schema, which is acceptable for a classroom prototype but should be replaced by non-destructive migrations before production.

5. Application flow

Secure Notes application flow

Validation happens before SQLite access; settings remain in SharedPreferences



LoginActivity first checks SessionManager. If a session exists, HomeActivity opens. Otherwise, the login listener reads fields and calls validation functions. The function returns before any database work when a field is blank or malformed. A successful login stores only the user ID in private session preferences.

HomeActivity displays NotesFragment by default. Bottom navigation replaces it with AccountFragment when requested. NotesFragment loads current-user records into NoteAdapter. The adapter inflates item_note.xml and uses a holder to recycle row bindings. The FAB launches AddNoteActivity and the Activity Result callback refreshes the ListView after a successful insert.

AccountFragment loads the current user's full name and email. SettingsActivity maps three radio-button choices to AppCompatActivity modes and persists the selected string. Logout clears the session and task history.

6. User-interface and creativity

The interface uses a restrained blue-and-gold palette, generous spacing, rounded Material cards, empty-state guidance, password visibility toggles, input error messages, an extended FAB, and subtle fragment fade transitions. Day/night color resources support readable contrast. Touch targets are at least 48 dp where practical.

7. Resource organization

- layout — activities, fragments, and item_note row
- values/strings.xml — displayed language
- values/colors.xml and values-night/colors.xml — palette alternatives
- values/styles.xml and themes.xml — reusable design rules
- values/dimens.xml — spacing, sizes, and corners
- drawable — vector icons and shape backgrounds
- menu — bottom-navigation destinations
- anim — fragment transitions

Layouts reference `@string`, `@color`, `@dimen`, `@style`, and theme attributes rather than embedding display values. This matches the course guidance on maintainability, localization, device adaptation, and generated R identifiers.

8. Testing plan

- Reject empty and malformed login input before querying.
- Reject duplicate accounts and mismatched passwords.
- Verify incorrect credentials do not start HomeActivity.
- Verify one account cannot see another account's notes.
- Verify empty note fields are rejected and valid notes appear newest-first.
- Verify all three theme choices survive application restart.
- Verify logout prevents Back from reopening private screens.
- Rotate each screen and check for crashes or lost navigation state.

9. Limitations and future improvements

The app is an offline classroom prototype. Future work should add encrypted note bodies, biometric unlock, ViewModel/repository separation, non-destructive database migrations, accessibility testing, unit/UI tests, and optional secure cloud synchronization.

10. Team demonstration plan

Recommended order: sign up; intentionally submit an empty login; log in; add two notes; explain the custom adapter and recycled row; long-press a note; navigate to account; switch among system/light/dark themes; show the privacy statement; log out. Each member should explain one SQL operation, one listener, one validation function, and one resource file.

11. Conclusion

Secure Notes satisfies the required authentication, navigation, notes, settings, and resource-organization features. Its design demonstrates the Android data and storage topics emphasized in BSD 324 while adding defensive input validation, password hashing, per-user data isolation, background database access, and a coherent Material interface.

Appendix: key source files

- data/DatabaseHelper.kt — schema and CRUD
- util/Validators.kt — login/sign-up input rules
- util/PasswordHasher.kt — salted PBKDF2
- ui/NoteAdapter.kt — ListView binding and recycling
- ui/NotesFragment.kt — FAB, loading, deletion
- util/ThemeManager.kt — SharedPreferences and AppCompatActivity
- res/values/* — strings, colors, styles, themes, dimensions